

AI Planning — STRIPS, Partial-Order Planning, State Variables & CSP

Concepts for Q3 (Evil Cat Robot), Q4 (Gorilla CSP), 2025 P7 Q2 (Evil Robot cave)

These three questions all live in the **representation** half of the planning syllabus: how you *describe* a planning problem (STRIPS, state variables), how you *solve* it without committing to an order (partial-order planning), and how you *translate* it into another formalism (CSP). State every definition as a standalone sentence before you use it — that is where the marks are.

1. STRIPS representation

(Q3, first bullet)

The cat/owner problem was given in *situation calculus* (fluents indexed by a situation term, with successor-state and frame axioms). STRIPS drops the explicit logic and instead describes each action by what it **adds** and **deletes**. To convert: each fluent $F(\bar{x}, s)$ becomes a timeless predicate $F(\bar{x})$; each action's successor-state axiom becomes an add-list and a delete-list.

STRIPS problem = $\langle$ states, operators, init, goal $\rangle$

- **State** — a *set of ground positive literals*. **Closed-world assumption**: any literal not in the set is false.
- **Operator (action schema)** $A(\bar{x})$ has

$\text{PRE}(a)$ = conjunction of literals, $\text{ADD}(a)$, $\text{DEL}(a)$ = lists of literals.

- **Initial state** — a complete set of literals true at the start.
- **Goal** — a conjunction of literals (need not mention every fluent).

Applicability & result (the bit examiners want stated precisely). Action a is applicable in state s iff $\text{PRE}(a)^+ \subseteq s$ and $\text{PRE}(a)^- \cap s = \emptyset$. The resulting state is

$$s' = (s \setminus \text{DEL}(a)) \cup \text{ADD}(a).$$

Only listed literals change — this is the **STRIPS assumption**, and it is what removes the need for explicit frame axioms.

Writing the cat problem in STRIPS. For *every* action in the situation-calculus version, give a schema, e.g. in the standard form:

```
Action(move(x,y),
  Pre: At(x) ∧ Adjacent(x,y),
  Add: At(y),
  Del: At(x) )
```

Use *parameters* (x, y) so one schema covers many ground actions; keep static facts (e.g. ADJACENT) as preconditions. The deliverable is: the set of operator schemas + the init literal set + the goal conjunction — exactly the input a partial-order planner expects.

2. Partial-order planning (POP)

(Q3, second bullet)

POP works backwards from the goal, adding actions and committing to an ordering **only when there is a causal reason** (least commitment). The output is a partial order, not a sequence.

A plan is a 4-tuple $\langle A, O, L, B \rangle$

- A — set of actions, including two dummies: START (no preconditions; its effects = the initial state) and FINISH (no effects; its preconditions = the goal).
- O — ordering constraints $A_i \prec A_j$ (“ A_i before A_j ”).
- L — causal links $A_i \xrightarrow{p} A_j$: “ A_i achieves precondition p of A_j ”.
- B — binding constraints on the schema variables.

Open precondition / threat / resolution

- An **open precondition** is a precondition with no causal link supporting it yet.
- A causal link $A_i \xrightarrow{p} A_j$ is **threatened** by an action A_k if $\neg p \in \text{ADD/effects of } A_k$ and A_k could be ordered between A_i and A_j . (A_k is a *clobberer*.)
- Resolve a threat by adding an ordering constraint: **demotion** $A_k \prec A_i$ (push the clobberer before the link) or **promotion** $A_j \prec A_k$ (push it after).

Algorithm (POP). Start with $A = \{\text{START}, \text{FINISH}\}$, $O = \{\text{START} \prec \text{FINISH}\}$, and FINISH’s preconditions open. Repeat:

1. Pick an open precondition p of some action A_j .
2. Choose an action A_i achieving p — either already in A or a new instance of a schema. Add the causal link $A_i \xrightarrow{p} A_j$, add $A_i \prec A_j$ (and $\text{START} \prec A_i \prec \text{FINISH}$ for new actions).
3. For every threat to any causal link, resolve it by promotion or demotion; if neither is consistent, backtrack.

Terminate when there are **no open preconditions** and $O \cup B$ is **consistent** (no cycle). The result is a *complete, consistent* partial-order plan.

Why POP — least commitment

n mutually independent actions have $n!$ total orders but POP keeps them as *one* unordered set, exploring none of the permutations. It only orders a pair when a causal link would otherwise be clobbered.

“How many specific plans can be extracted?”

A **specific (totally ordered) plan** is any **linearisation** of the partial order: a topological sort of the action DAG consistent with O . So the count is the **number of topological orderings (linear extensions)** of the final partial order.

Method: draw the DAG from O ; any action with all predecessors already placed may go next; multiply out the choices. If the solution fully orders the actions there is exactly **1**; each pair left genuinely unordered multiplies the count. State the number *and* justify it by exhibiting the unordered actions.

3. State-variable representation

(2025 P7 Q2 a, b)

Propositional STRIPS uses many mutually-exclusive booleans ($\text{At}(1, 1, 1)$, $\text{At}(1, 1, 2)$, ...). The **state-variable** (SAS^+ / multi-valued) representation replaces each such set by *one* finite-domain variable — far more compact and the natural bridge to a CSP.

Elements of the state-variable representation (Q2a)

- A set of **state variables** $V = \{v_1, \dots, v_n\}$, each with a finite **domain** D_v .
- A **state** is a total assignment giving every v a value in D_v .
- **Actions**: precondition = a set of *tests* $v = d$; effect = a set of *assignments* $v := d'$.
- **Initial state** = a full assignment; **goal** = a *partial* assignment (constrains some variables only).

For the cave (Q2b examples). Variables such as $\text{LOC} \in \{1, \dots, 10\}^3$, $\text{HASKEY} \in \{\top, \perp\}$, $\text{GOLDFOUND} \in \{\top, \perp\}$.

- *Traps / key dispensers* — fixed facts about the map, so model as **rigid relations** (see §4): $\text{TRAP}(x, y, z)$, $\text{KEYDISPENSER}(x, y, z)$, not state variables.
- *Obtaining a key* — action: $\text{PRE} : \text{KEYDISPENSER}(\text{LOC})$; effect: $\text{HASKEY} := \top$.
- *Escape from prison* — action: $\text{PRE} : \text{LOC} = (10, 10, 10) \wedge \text{HASKEY} = \top$; effect: $\text{LOC} := (1, 1, 1)$.

4. Rigid vs. fluent relations

(2025 P7 Q2 c)

Definition

A **fluent** is a relation/variable whose truth *changes between states* as actions are applied (LOC, HASKEY). A **rigid relation** is *fixed for the whole problem instance* and never changed by any action — the static structure of the world (walls, which cells are adjacent in each direction, magic-path links, trap/dispenser positions).

The key trick (what Q2c is testing). Put a rigid relation *in the precondition* of a single action schema so that one schema specialises into exactly the legal concrete actions. Define a directional adjacency rigid relation, e.g. $\text{RIGHT}(\ell, \ell')$ holding for just those pairs where ℓ' is the cell to the right of ℓ and there is no wall:

$\text{Action}(\text{goRight}, \text{Pre: Loc} = \ell \wedge \text{Right}(\ell, \ell'), \text{Effect: Loc} := \ell')$

- **Four descriptions, one shape:** GOLEFT , GORIGHT , GOFORWARD , GOBACK differ *only* in which rigid relation (LEFT / RIGHT / FORWARD / BACK) they consult — so you reuse a single effect schema four times.
- **Walls handled for free:** a move into a wall simply has *no* rigid tuple, so the action is not applicable — no special case needed.
- **Can't escape prison by moving:** give the prison cell *no outgoing* directional tuples in the rigid relations. Then no GO^* action applies there; leaving prison is only possible via the dedicated escape action (which requires the key). This is exactly the “account for the fact that you can't escape just by moving” clause.

5. Constraint satisfaction problems

(Q4 gorilla; 2025 P7 Q2 d)

CSP = $\langle X, D, C \rangle$

- **Variables** $X = \{x_1, \dots, x_n\}$.
- **Domains** $D = \{D_1, \dots, D_n\}$, $x_i \in D_i$ (finite).
- **Constraints** C : each $c \in C$ ranges over a subset of X and lists the *allowed* value-combinations.

A **solution** is a complete assignment satisfying every constraint. Constraints over two variables are *binary*; the **constraint graph** has an edge per binary constraint. **Arc consistency** (AC-3) prunes domain values with no support across an arc, before/while searching.

Q4 — encoding “buy a gorilla and attach it to the spire” as a CSP

The question is open (“add whatever actions, relations ...you feel appropriate”); what is marked is that you produce well-formed *variables + domains + constraints*. Using the given domains:

$$\begin{aligned} D_1 &= \{\text{climber}\}, & D_3 &= \{\text{rope}, \text{gorilla}, \text{firstAidKit}\}, \\ D_2 &= \{\text{home}, \text{jokeShop}, \text{hardwareStore}, \text{spire}\}. \end{aligned}$$

Introduce variables for *where each item is acquired* and the *final location*, e.g. $\text{BUYLOC}_{\text{rope}}, \text{BUYLOC}_{\text{gorilla}} \in D_2$ and $\text{FINISHAT} \in D_2$, plus the agent $\text{WHO} \in D_1$. Constraints (rigid “sold-at” facts plus a goal constraint):

- *availability* (unary): $\text{BUYLOC}_{\text{gorilla}} = \text{jokeShop}$, $\text{BUYLOC}_{\text{rope}} = \text{hardwareStore}$ — each item only sold at its shop.
- *goal* (the attach action’s precondition as a constraint): $\text{FINISHAT} = \text{spire}$, and the climber must *carry rope* and *gorilla* (and, sensibly, *firstAidKit*) — encode “carried” as boolean variables forced true.
- *resource/ordering* constraints if you model time-steps: you can only carry an item after visiting its shop.

State clearly: variables, their domains, and each constraint as a relation — that is a complete CSP encoding. (If you add a time index, it becomes the bounded-planning encoding below.)

Q2d — modelling goRight as part of a CSP (planning $\rightarrow$ CSP)

Bounded-horizon planning as a CSP

Fix a plan length T . Create, for each step $t \in \{0, \dots, T\}$, a copy of every state variable v^t , and for each $t \in \{0, \dots, T-1\}$ an action variable ACT^t ranging over the actions. Constraints:

- **init** $v^0 = \text{initial value}$; **goal** $v^T = \text{goal value}$.
- **precondition**: $\text{ACT}^t = a \Rightarrow \text{PRE}(a)$ holds in the t -layer.
- **effect**: $\text{ACT}^t = a \Rightarrow$ the $(t+1)$ -layer matches a ’s assignments.
- **frame**: variables not in a ’s effect satisfy $v^{t+1} = v^t$.

A solution assignment *is* a length- T plan; a solver searches over T .

For `goRight` specifically, the constraint tying consecutive layers is

$$\text{ACT}^t = \text{goRight} \implies \text{RIGHT}(\text{LOC}^t, \text{LOC}^{t+1}) \wedge \text{HASKEY}^{t+1} = \text{HASKEY}^t \wedge \dots$$

i.e. the rigid `RIGHT` relation constrains LOC^{t+1} to the right-neighbour of LOC^t (and is unsatisfiable at a wall), while the frame conjuncts copy every other variable forward. This is why the state-variable form is convenient: each action is just a constraint relating layer t to layer $t+1$.

Exam checklist

1. **STRIPS:** state = set of literals (closed-world); action = $\langle \text{PRE}, \text{ADD}, \text{DEL} \rangle$; result $s' = (s \setminus \text{DEL}) \cup \text{ADD}$. To convert from situation calculus: fluent $\rightarrow$ predicate, successor-state axiom $\rightarrow$ add/delete lists, frame axioms vanish.
2. **POP:** plan = $\langle A, O, L, B \rangle$ with START/FINISH; causal links; threats resolved by *promotion/demotion*; stop when no open preconditions and O acyclic.
3. **#specific plans** = number of *linearisations* (topological sorts) of the partial order. 1 if fully ordered; more for each unordered pair — justify by naming the unordered actions.
4. **State variables:** finite-domain variables; state = full assignment; goal = partial assignment. More compact than booleans.
5. **Rigid vs fluent:** rigid = never changes (map/walls/adjacency); fluent = changes with actions. Put a rigid relation in the precondition to get *four* GO* schemas from one shape, handle walls automatically, and block prison-escape-by-moving.
6. **CSP:** $\langle X, D, C \rangle$; solution = satisfying complete assignment. Planning $\rightarrow$ CSP: layer the state variables over T steps; each action is a constraint linking layer t to $t+1$ (precondition + effect + frame).